



Unidad 3

Polimorfismo

Prof: Lic. Rosales Pablo (prosales@unpata.edu.ar)
Materia: Programación Orientada a Objetos
Año: 2026



La pre clase, en dos preguntas



- El cierre de caja del local: listar todos los cobros del día y sumar cuanto se acredita.
- Estaba escrito como tres bucles idénticos, uno por medio de pago ¿Que pasa el dia que se acepten transferencias?
- Las clases 5 y 6 ya dejaron todo lo necesario para que ese tercer bucle no exista ¿Qué es lo que falta?

Falta poder escribir un solo bucle que no pregunte de qué tipo es cada pago.

El problema: tres bucles iguales



```
for (PagoDebito pago : debitos) {  
    total += pago.montoAcreditado();  
}  
  
for (PagoCredito pago : creditos) {  
    total += pago.montoAcreditado();  
}  
  
for (PagoQR pago : qrs) {  
    total += pago.montoAcreditado();  
}
```

El cierre de caja conoce por nombre a cada medio de pago. Cada medio nuevo lo obliga a cambiar.

Una variable de un tipo, un objeto de otro



```
Pago pago = new PagoQR(1, venta, "PAGO;lapampa.kiosco;ARS");  
  
pago.mostrarComprobante();    // compila: Pago lo declara  
pago.armarSolicitudCobro();   // NO compila, aunque el objeto sea un PagoQR
```

Tipo estático: el de la variable. Tipo dinámico: el del objeto. La referencia solo ve lo que su tipo declara.

El compilador mira la variable.

La ejecución mira el objeto.



Que decide cada uno



El compilador

- Verifica que el método **exista** en el tipo de la variable.
- Si `Pago` no lo declara, no compila. No le importa que el objeto lo tenga.

La ejecución

- Elige **cuál versión** se ejecuta, mirando el objeto real.
- Se llama **enlace dinámico**, y es lo que le da sentido a sobrescribir.

El mismo mensaje, distinta respuesta, y quien lo envía no necesita saber quien lo recibe.

El cierre de caja, otra vez



```
public static double cerrarCaja(ArrayList<Pago> pagos) {  
    double total = 0;  
    for (Pago pago : pagos) {  
        pago.mostrarComprobante();           // cada uno imprime el suyo  
        total += pago.montoAcreditado();     // cada uno retiene lo suyo  
    }  
    return total;  
}
```

Ningun if, ningun nombre de subclase. El día que entre PagoTransferencia, este método no se toca.

Y con interfaces es exactamente lo mismo



- El `anularTodo (ArrayList<Anulable> operaciones)` de la clase pasada ya era polimórfico, sin haberlo nombrado.
- Recorria pagos con QR, débitos y ventas sin distinguirlos, y cada `anular ()` hacia algo distinto.
- La única diferencia: ahí el tipo común venía de un contrato, no de una superclase.

Herencia o interfaz, el mecanismo es el mismo: programar contra la abstracción.

Los dos polimorfismos



Sobrecarga (estático)

- Varios métodos con el **mismo nombre** en la misma clase.
- Cambia la lista de parámetros.
- Decide el **compilador**, mirando los argumentos.
- Ejemplo: `System.out.println`

Sobrescritura (dinámico)

- El mismo método en la superclase y en la subclase.
- La firma es **idéntica**.
- Decide la **ejecución**, mirando el objeto.
- Ejemplo: `mostrarComprobante()`.

Los dos se llaman polimorfismo. El que sostiene el diseño orientado a objetos es el segundo.

Dos cosas que NO son polimórficas



- Los **atributos** no se sobrescriben. Si el padre y el hijo declaran el mismo nombre, conviven los dos, y el que se usa depende del tipo de la **variable**.
- Otra razón más para lo mismo de siempre: los atributos van `private`.
- Los métodos **static** tampoco. Pertenecen a la clase, no al objeto. La subclase no los sobrescribe: los tapa.

El enlace dinámico vale para los métodos de instancia, y solo para ellos.

La actividad de hoy



- Se sigue con la tienda. Ninguna clase del catálogo se toca: el unico archivo que se modifica es `Tienda.java`.
- **Parte 1:** contar cuantos lugares hay que tocar para sumar un tipo de producto nuevo.
- **Parte 2:** las tres listas y los tres bucles pasan a ser un `ArrayList<Producto>` y un bucle.
- **Parte 3:** escribir `Indumentaria` y sumarla al catálogo. Contar cuantos metodos hubo que cambiar.